

```

class A:
    x = 10
    def __init__(self):
        print('hello')

a = A()
hello
a
<__main__.A at 0x2b5495eaf00>

```

```

class B:
    x = 10
    def __init__(self):
        print('hello')
    def __repr__(self):
        print('Class B')

```

```

b = B()
hello
b
Class B

```

```

-----
-----
TypeError                                Traceback (most recent call
last)

```

```

File ~\AppData\Local\Programs\Python\Python312\Lib\site-packages\
IPython\core\formatters.py:770, in PlainTextFormatter.__call__(self,
obj)

```

```

    763 stream = StringIO()
    764 printer = pretty.RepresentationPrinter(stream, self.verbose,
    765     self.max_width, self.newline,
    766     max_seq_length=self.max_seq_length,
    767     singleton_pprinters=self.singleton_pprinters,
    768     type_pprinters=self.type_pprinters,
    769     deferred_pprinters=self.deferred_pprinters)
--> 770 printer.pretty(obj)
    771 printer.flush()
    772 return stream.getvalue()

```

```

File ~\AppData\Local\Programs\Python\Python312\Lib\site-packages\
IPython\lib\pretty.py:412, in RepresentationPrinter.pretty(self, obj)

```

```

    401         return meth(obj, self, cycle)
    402         if (
    403             cls is not object
    404             # check if cls defines __repr__

```

```

(...)      410                                and callable(_safe_getattr(cls,
"__repr__", None))
      411                                ):
--> 412                                return _repr_pprint(obj, self, cycle)
      414                                return _default_pprint(obj, self, cycle)
      415 finally:

```

File ~\AppData\Local\Programs\Python\Python312\Lib\site-packages\IPython\lib\pretty.py:787, in _repr_pprint(obj, p, cycle)

```

      785 """A pprint that just redirects to the normal repr
function."""

```

```

      786 # Find newlines and replace them with p.break_()
--> 787 output = repr(obj)
      788 lines = output.splitlines()
      789 with p.group():

```

TypeError: __repr__ returned non-string (type NoneType)

```

class B:
    x = 10
    def __init__(self):
        print('hello')
    def __repr__(self):
        return('Class B')

```

```
b = B()
```

```
hello
```

```
b
```

```
Class B
```

```

class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade
    def __repr__(self):
        return(f'{self.name} Object')

```

```
s1 = Student('Anubhav', 'A')
```

```
s2 = Student('Jaya', 'A')
```

```
s1
```

```
<__main__.Student at 0x2b548db0920>
```

```
s2
```

```
<__main__.Student at 0x2b5495e95e0>
```

s1

Anubhav Object

```
class A:
    x = 76
    # def __init__(self):
    #     print('hello')
    @staticmethod
    def abcd():
        return('abcd method')
```

A.abcd()

'abcd method'

```
class B:

    def abcd(self):
        return('hello')
```

b = B()

b.abcd()

'hello'

```
class D:
    # @staticmethod
    def abcd():
        return('static method')
```

D.abcd()

'static method'

d = D()

d.abcd()

TypeError Traceback (most recent call
last)

Cell In[52], line 1

----> 1 d.abcd()

TypeError: D.abcd() takes 0 positional arguments but 1 was given

```
class Naved:
    def ab(self):
        return f'naved method - instance'
```

```

class Anubhav:
    def abcd():
        return f'Anubhav method - static'

n = Naved()
a = Anubhav()

n.ab()
'naved method - instance'

Anubhav.abcd()
'Anubhav method - static'

class Email:
    @staticmethod
    def is_valid(email):
        return ('@' in email and '.' in email)

Email.is_valid('nashitabc.com')
False

e = Email()
e.is_valid('nashit@abc.com')
True

class Account:
    def __init__(self, name, balance = 0):
        self.name = name
        self.balance = balance
    def deposit(self, amount):
        # self.amount=amount
        self.balance += amount
    def check(self):
        return f'{self.name}, your balance is {self.balance}'

nashit = Account('Nashit')
nashit.check()
'Nashit, your balance is 0'

nashit.deposit(500)
nashit.check()
'Nashit, your balance is 500'

```